Approfondimento: la cache della CPU

Perché l'ordine di accesso alla memoria è importante

Perché è utile conoscerla?

Quando scriviamo un programma, possiamo valutare il lavoro che svolge contando le operazioni eseguite. Tuttavia, il tempo di esecuzione non dipende soltanto dal numero di operazioni: conta anche il modo in cui il programma accede ai dati in memoria.

Questo aspetto è particolarmente importante quando si elaborano grandi quantità di dati, per esempio matrici, immagini o dati usati nelle simulazioni scientifiche.

Vedi codice **efficienza.c** nel pacchetto sugli Array.

1 RAM e cache: un'idea intuitiva

La CPU esegue le istruzioni del programma e deve recuperare dalla memoria i dati su cui lavorare. La RAM può contenere molti dati, ma è più lenta rispetto alla velocità con cui la CPU può elaborare informazioni. Per ridurre le attese, i processori usano piccole memorie molto veloci chiamate **cache**.

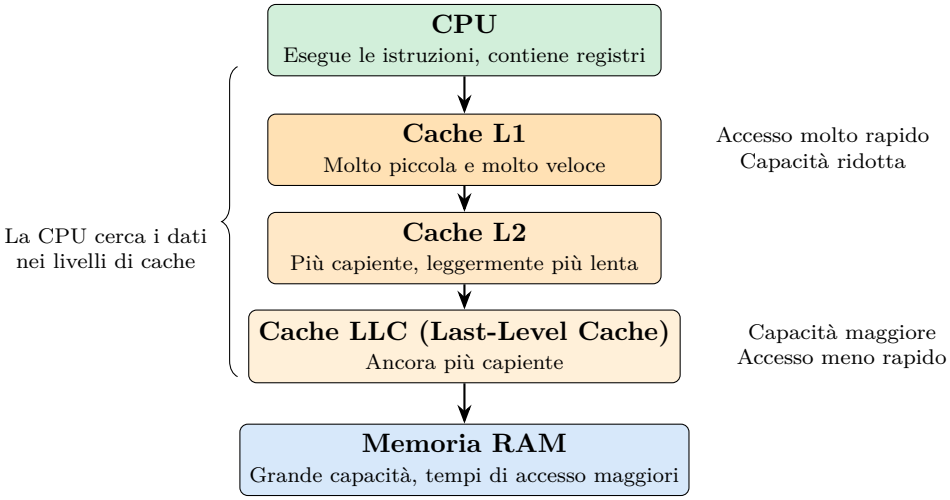
1.1 Che cos'è la cache della CPU?

La cache conserva copie di dati che sono stati utilizzati di recente o che si trovano vicino a dati appena richiesti. In questo modo, se la CPU ha nuovamente bisogno di un dato già presente nella cache, può recuperarlo senza doverlo leggere nuovamente dalla RAM.

I livelli di cache. Nei processori moderni esistono generalmente più livelli di cache, caratterizzati da dimensioni e velocità differenti:

- **L1 (Level 1):** è generalmente la cache più piccola e veloce, situata molto vicino alle unità di esecuzione del processore. Spesso è suddivisa in una parte per le istruzioni e una per i dati.
- **L2 (Level 2):** è generalmente più capiente della L1, ma presenta tempi di accesso maggiori.
- **LLC (Last-Level Cache):** è l'ultimo livello della gerarchia di cache, prima della memoria principale. Nei processori moderni corrisponde spesso alla L3 ed è generalmente più capiente dei livelli precedenti. Può essere condivisa tra più core.

La gerarchia è progettata per combinare la velocità delle cache più piccole con la maggiore capacità della memoria RAM.



La figura rappresenta una gerarchia concettuale, non il percorso obbligatorio di ogni singolo accesso. Quando la CPU richiede un dato, se questo è già presente in una delle cache vicine, può recuperarlo rapidamente. Se non è presente, l'hardware può doverlo recuperare da un altro livello della gerarchia, eventualmente dalla RAM, e copiarlo nella cache.

Quando il dato richiesto è già disponibile nella cache si parla di **cache hit**; quando non è presente si parla di **cache miss**. Un cache miss può comportare un'attesa per il recupero del dato da un livello di memoria più lento.

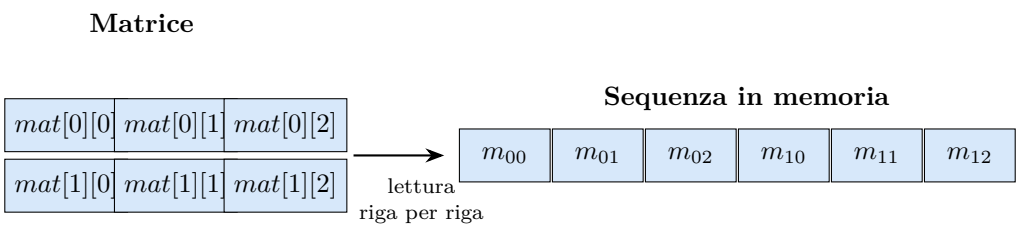
Questo meccanismo è particolarmente importante quando lavoriamo con array e matrici: accedere in sequenza a elementi vicini in memoria permette spesso di sfruttare meglio i dati già caricati nella cache, migliorando le prestazioni del programma.

2 Come sono disposti gli elementi di una matrice in C?

Consideriamo una matrice dichiarata così:

```c
int mat[2][3];
```

La matrice contiene 6 interi. In C, gli elementi sono disposti in memoria **riga per riga** (*row-major order*): prima tutti gli elementi della riga 0 e poi tutti quelli della riga 1.



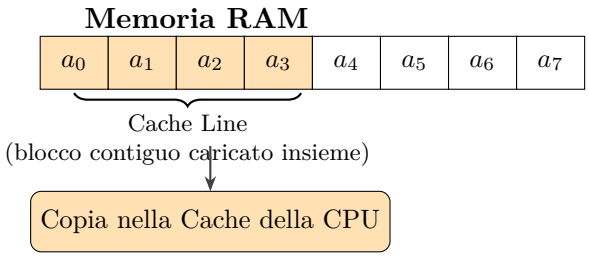
La rappresentazione a griglia è comoda per noi, ma in memoria gli elementi formano una sequenza contigua. Per questo, dopo `mat[0][2]` si trova immediatamente `mat[1][0]`, poi `mat[1][1]` e infine `mat[1][2]`.

Nota: il disegno mostra l'ordine degli elementi, non indirizzi reali. La dimensione di un `int` dipende dall'architettura; spesso è di 4 byte.

3 Che cos'è una cache line?

Quando la CPU richiede un dato alla RAM, non trasferisce mai un solo valore isolato (ad esempio un singolo intero da 4 byte). La memoria viene mossa in blocchi contigui di dimensione fissa (solitamente 64 byte nei processori moderni), chiamati **linee di cache** (*cache line*).

Questo significa che, anche se il programma chiede un solo elemento, la cache si riempie automaticamente anche con i dati che si trovano immediatamente prima e dopo in memoria.

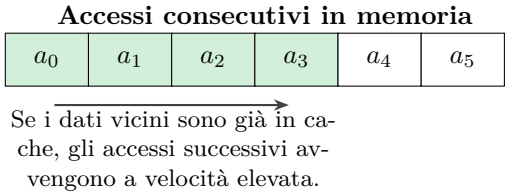


Se il programma, subito dopo aver letto il primo dato, ha bisogno dei valori vicini (come a_1, a_2, a_3), non dovrà più attendere i tempi lenti della RAM: li troverà già pronti e velocissimi all'interno della cache.

4 La località spaziale

Questo comportamento è un esempio di **località spaziale**: quando un programma accede a un dato, è probabile che in breve tempo acceda anche ad altri dati che si trovano vicino a esso in memoria.

Per esempio, attraversando una riga di una matrice, il programma visita elementi contigui. Se alcuni di essi sono già stati caricati nella stessa linea di cache, possono essere riutilizzati senza dover recuperare ogni volta un nuovo blocco dalla RAM.



La cache non garantisce che ogni accesso sia veloce: il risultato dipende anche dai dati già presenti, dalla dimensione della cache e dal comportamento complessivo del programma. Il principio, però, aiuta a capire perché l'ordine degli accessi può influenzare le prestazioni.

5 Attraversare una matrice per righe o per colonne

Riprendiamo due modi di visitare tutti gli elementi di una matrice. Entrambi eseguono $R \times C$ operazioni, ma seguono ordini di accesso diversi (puoi verificare questo comportamento nel codice **efficienza.c**).

Versione A: per righe

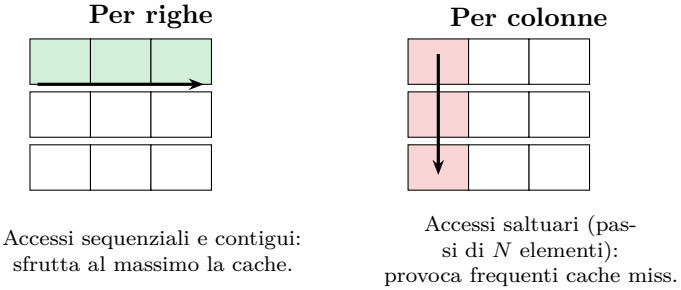
```
for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        somma += mat[i][j];
    }
}
```

Il programma completa una riga prima di passare alla successiva. Gli accessi seguono l'ordine naturale in cui gli elementi sono disposti in memoria (*row-major order*).

Versione B: per colonne

```
for (int j = 0; j < C; j++) {
    for (int i = 0; i < R; i++) {
        somma += mat[i][j];
    }
}
```

Il programma visita prima la prima colonna, poi la seconda e così via. Poiché gli elementi di una stessa colonna sono separati in memoria da un'intera riga, gli accessi saltano continuamente in avanti e indietro, risultando meno efficienti per la cache.



Accessi sequenziali e contigui: sfrutta al massimo la cache.

Accessi saltuari (passi di N elementi): provoca frequenti cache miss.

Come dimostrato da test reali su matrici di grandi dimensioni (es. 10000×10000), l'attraversamento per righe sfrutta la località spaziale risultando drasticamente più veloce, mentre quello per colonne costringe la CPU ad attese continue per recuperare i dati dalla RAM.

6 Che cosa dobbiamo ricordare?

- La **cache** è una memoria piccola e veloce che riduce il tempo necessario per accedere ai dati.
- Una **linea di cache** contiene un blocco di dati vicini in memoria caricati simultaneamente.
- La **località spaziale** fa sì che, accedendo a un dato, i valori vicini siano già pronti per l'uso.
- In C, le matrici sono in formato *row-major*: scorrerle per righe rispetta l'hardware, scorrerle per colonne spreca prestazioni.

L'idea centrale è che due programmi possono eseguire lo stesso numero esatto di operazioni logiche, ma impiegare tempi radicalmente diversi in base a **come** scelgono di organizzare gli accessi alla memoria.